



UNIVERSIDAD DE MÁLAGA



Graduado en Ingeniería Informática

Parametrization of the quantum approximate optimization algorithm through machine learning for combinatorial problems

Parametrización del algoritmo de optimización aproximado cuántico para problemas combinatorios mediante aprendizaje máquina

Realizado por
Mario Quiñones Madrona

Tutorizado por
Gabriel Jesús Luque Polo
Zakaria Abdelmoiz Dahi

Departamento
Lenguajes y Ciencias de la Computación

MÁLAGA, junio 2025



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADUADO EN INGENIERÍA INFORMÁTICA

**Parametrization of the quantum approximate optimization
algorithm through machine learning for combinatorial
problems**

**Parametrización del algoritmo de optimización aproximado
cuántico para problemas combinatorios mediante
aprendizaje máquina**

Realizado por
Mario Quiñones Madrona

Tutorizado por
Gabriel Jesús Luque Polo
Zakaria Abdelmoiz Dahi

Departamento
Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2025

Fecha defensa: julio de 2025

Abstract

Quantum computing offers transformative potential for solving complex optimization problems, but access to quantum hardware remains severely limited due to technical constraints and high operational costs. The Quantum Approximate Optimization Algorithm (QAOA) is particularly resource-intensive, requiring extensive quantum processor time for parameter optimization through repeated circuit executions.

While existing approaches address individual aspects the QAOA’s efficiency – such as parameter initialization strategies or circuit-depth reduction – this project introduces an integrated machine learning framework to predict optimal QAOA parameters classically, minimizing quantum resource dependency.

This work advances quantum resource efficiency through a multi-faceted machine learning framework designed to parametrize the QAOA. The core innovation lies in the development and comparative evaluation of different supervised machine learning techniques—including Gradient Boosting, Bayesian Neural Networks and Graph Neural network- trained to predict optimal QAOA parameters. These models were rigorously trained and validated on both MaxCut instances and circuit level data in order to enable a hybrid workflow where ML-predicted parameters seed quantum optimizers, reducing quantum circuit executions.

Keywords: Quantum Computing, Supervised Machine Learning, QAOA

Resumen

La computación cuántica ofrece un potencial transformador para resolver problemas complejos de optimización, pero el acceso al hardware cuántico sigue estando severamente limitado debido a restricciones técnicas y altos costos operativos. El Algoritmo Cuántico Aproximado de Optimización (QAOA) es particularmente intensivo en recursos, ya que requiere un tiempo de procesador cuántico extensivo para la optimización de parámetros mediante ejecuciones repetidas de circuitos.

Si bien los enfoques existentes abordan aspectos individuales de la eficiencia del QAOA –como estrategias de inicialización de parámetros o reducción de la profundidad del circuito–, este proyecto introduce un marco de aprendizaje automático integrado para predecir clásicamente los parámetros óptimos del QAOA, minimizando la dependencia de los recursos cuánticos.

Este trabajo avanza en la eficiencia de los recursos cuánticos mediante un marco de aprendizaje automático multifacético diseñado para parametrizar el Algoritmo Cuántico Aproximado de Optimización (QAOA). La innovación central radica en el desarrollo y evaluación comparativa de diferentes técnicas de aprendizaje automático supervisado—incluyendo Gradient Boosting (Potenciación del Gradiente), Redes Neuronales Bayesianas y Redes Neuronales de Grafos—entrenadas para predecir los parámetros óptimos del QAOA. Estos modelos fueron rigurosamente entrenados y validados tanto en instancias del problema MaxCut como en datos a nivel de circuito, con el fin de habilitar un flujo de trabajo híbrido donde los parámetros predichos por el aprendizaje automático inician los optimizadores cuánticos, reduciendo así las ejecuciones de circuitos cuánticos.

Palabras Clave: Computación cuántica, aprendizaje máquina supervisado, QAOA

Contents

1	Introduction	9
1.1	Motivation	9
1.2	Objectives	10
1.3	Technologies used	11
1.3.1	Numpy	11
1.3.2	Qiskit	11
1.3.3	Pandas	11
1.3.4	Scikit-learn	12
1.3.5	Pytorch	12
1.4	Structure of the document	12
2	Background	15
2.1	Quantum Computing	15
2.2	Gate-base Quantum Computing	16
2.2.1	Hadamard Gate (H)	17
2.2.2	Rotation Gates	18
2.2.3	Controlled-NOT (CNOT) Gate	18
2.2.4	Pauli matrices	18
2.3	Quadratic Unconstrained Binary Optimization Problems	19
2.3.1	Definition and Formulation	20
2.3.2	Key Properties	20
2.3.3	Applications and Extensions	20
2.3.4	Applications in Quantum Computing	20
2.3.5	MaxCut Problem	21
2.4	Quantum Approximate Optimization Algorithm	22
2.4.1	Variational Quantum Algorithms	22
2.4.2	Overview of QAOA	23
2.4.3	QAOA Framework	25

2.5	Machine Learning	28
2.5.1	Supervised Machine Learning	29
2.5.2	Neural Networks	30
2.5.3	Graph Neural Networks	31
2.5.4	Gradient Boosting	32
3	Proposal Overview	35
4	Experimental Design	39
4.1	Dataset	40
4.1.1	QUBO Formulation for MaxCut	40
4.1.2	Pauli Hamiltonian Conversion	40
4.1.3	QAOA Circuit Preparation and Execution	41
4.1.4	Hybrid Quantum-Classical Optimization	41
4.1.5	Dataset Curation for Empirical Analysis	41
4.2	XGBoost	42
4.2.1	Model Architecture and Training Protocol	42
4.2.2	Feature Engineering and Data Representation	43
4.2.3	Evaluation Metrics	43
4.2.4	Experimental Setup	44
4.3	Multilayer Perceptron	44
4.3.1	Neural Network Architecture	44
4.3.2	Training Configuration	45
4.3.3	Hyperparameter Optimization	45
4.3.4	Evaluation Metrics	45
4.3.5	Experimental Execution	45
4.3.6	Computational Considerations	46
4.4	Graph Neural Networks	46
4.4.1	Architecture Design	46
4.4.2	Graph Data Construction	47
4.4.3	Training Protocol	47
4.4.4	Evaluation Metrics	48

4.4.5	Computational Environment	48
4.5	Circuit Level Data	48
5	Results and comparison	51
5.1	XGBoost	51
5.2	Multilayer Perceptron	53
5.3	Graph Neural Networks	54
5.4	Model comparison	56
5.5	Comparison with conventional QAOA approach	57
6	Conclusions and Futures Lines of Research	59
6.1	Conclusions	59
6.2	Future lines of Research	59
Appendix A Installation		
	Manual	67

1

Introduction

The purpose of this first chapter is to present the rationale for this work, define its objectives, identify the core software technologies relied upon during development, and describe the document's structure.

1.1 Motivation

Quantum computing is a growing field that seeks to solve complex problems more efficiently than classical computers [33]. To achieve this, multiple quantum algorithms have been developed that can offer significant advantages over traditional methods by leveraging the characteristics of quantum computers. Among these algorithms, the Quantum Approximate Optimization Algorithm (QAOA) [4] has proven to be one of the most promising proposals for tackling combinatorial optimization problems. QAOA works via a parameterized quantum circuit that is tuned to minimize a cost function and find near-optimal solutions.

However, a key challenge in implementing QAOA is selecting adequate parameters, as it requires running the circuit multiple times to find suitable configurations. Traditionally, this process has been addressed using classical optimization, which entails high computational costs. This becomes even more problematic given current quantum simulation and computational constraints. To reduce this complexity, various studies have explored the use of machine learning [32] to predict good parameters without the need for intensive optimization.

Despite these advances, opportunities still exist to improve the efficiency and accuracy of these strategies. Some deficiencies found in existing work include the limited use of machine learning models, with a predominance of unsupervised approaches. Furthermore, these models have been evaluated on a very small number of problems, raising doubts about their generality. Another significant deficiency lies in the modeling of the learning task itself, where the problem representation and choice of parameters may not be optimal. Similarly, some stud-

ies present an incomplete learning pipeline, omitting key stages such as feature selection and extraction, post-processing of results, or rigorous model validation across different scenarios.

1.2 Objectives

The objectives of this final degree thesis are specialized in quantum computing, a relatively new field in which few have experience. To avoid repeating explanations, the necessary concepts are presented in Chapter 2, which should be read first if you're not already familiar with quantum computing. The objective of this project is to develop a robust framework for optimizing QAOA parameters using machine learning techniques, addressing the critical challenge of efficient parameter selection in quantum optimization. Chapter 2 will establish foundational background knowledge, covering fundamental aspects of gate-based quantum computing, including qubit operations and circuit model execution. This will be followed by an introduction to Quadratic Unconstrained Binary Optimization (QUBO) problems and their role in combinatorial optimization. The theoretical framework of QAOA will be detailed, emphasizing its parameterized ansatz structure and the inherent difficulties in optimizing its energy landscape. Relevant machine learning methodologies for regression, feature extraction, and structured data processing will also be surveyed.

Building upon this groundwork, a machine learning approach will be implemented. The XGBoost will be employed for tabular regression using problem-instance features such as graph density and degree distributions. Multilayer perceptrons will be leveraged to identify high-dimensional patterns in continuous parameter spaces, while graph neural networks will exploit topological correlations between problem graphs and optimal parameters. Circuit-level feature engineering will incorporate descriptors such as number of qubits, circuit depth, and number of gates.

The study will conclude with guidelines for parameterization strategies, an analysis of current limitations in ML-QAOA integration, and future research directions for quantum-assisted machine learning.

1.3 Technologies used

Once the objectives have been outlined, we proceed to present the main software technologies that have enabled us to develop this project. The core implementation utilizes Python; the key modules employed are listed subsequently.

1.3.1 Numpy

NumPy (Numerical Python) is the foundational module for numerical computing in Python. It provides a powerful N-dimensional array object, `ndarray`, along with a suite of mathematical functions and tools for efficient computation, broadcasting, linear algebra, Fourier transforms, and random number generation. NumPy is optimized for performance and serves as the backbone for many scientific and data libraries, including Pandas, SciPy, and scikit-learn. Its efficiency and flexibility make it an essential tool for scientific computing, data analysis, and any application requiring fast numerical operations.

1.3.2 Qiskit

Qiskit (*Quantum Information Science Kit*) is an open-source quantum computing software development framework developed by IBM. It provides a comprehensive set of tools for designing, simulating, and executing quantum algorithms on both quantum simulators and real quantum hardware. Qiskit allows users to create quantum circuits using a high-level Python interface, offering modules for quantum circuit construction, algorithm development, and transpilation (process that converts the logical circuit into one executable on the target hardware, similar to the compilation process in traditional computers) and optimization for quantum devices.

1.3.3 Pandas

Pandas is an open-source Python module that provides high-performance, easy-to-use data structures and data analysis tools. Built on top of NumPy, it introduces two primary data types: `DataFrame` for two-dimensional labeled data and `series` for one-dimensional labeled data. Pandas simplifies data manipulation, cleaning, filtering, and transformation tasks, making it an essential tool for data analysis, preprocessing, and exploration. It integrates well with

other scientific computing libraries and is widely used in data science, finance, economics, and many other fields requiring structured data analysis.

1.3.4 Scikit-learn

Scikit-learn is a powerful, open-source machine learning module for Python that provides simple and efficient tools for data mining and data analysis. Built on top of *NumPy*, *SciPy*, and *matplotlib*, it includes a wide range of supervised and unsupervised learning algorithms such as classification, regression, clustering, and dimensionality reduction. With a consistent and user-friendly API, scikit-learn is designed to work seamlessly with numerical and scientific Python tools.

1.3.5 Pytorch

PyTorch is an open-source machine learning module for Python, developed to facilitate the construction and training of deep learning models. It features dynamic computation graphs and high-performance tensor operations, allowing for intuitive model development and real-time debugging. Widely used in academic research and industrial applications, **PyTorch** is particularly effective for tasks such as computer vision, natural language processing, and reinforcement learning.

1.4 Structure of the document

This document is organized into six main chapters, along with a final appendix, described as follows:

- **Chapter 1:** Introduction. This chapter presents the motivation for the work, the proposed objectives, the structure of the document, and the technologies used in developing the model. It establishes the relevance of quantum optimization in machine learning workflows.
- **Chapter 2:** Background. The theoretical framework is addressed, covering gate-based quantum computing fundamentals, QUBO (Quadratic Unconstrained Binary Optimization) problem formulation, and the principles of the Quantum Approximate Optimization Algorithm (QAOA).

- **Chapter 3:** Proporsal Overview. A summary of the proposed methodology for integrating machine learning model optimization pipelines into quantum techniques is presented.
- **Chapter 4:** Experimental Design. The experimental implementation is detailed, including the dataset used, configurations of classical models (XGBoost, Multilayer Perceptron, and Graph Neural Networks), and quantum circuit-level data processing.
- **Chapter 5:** Results and comparison. Experimental results are presented along with a comparative performance analysis of the evaluated techniques.
- **Chapter 6:** Conclusions and Futures Lines of Research. Conclusions derived from the study are discussed, and potential directions for future research in classical-quantum hybrid optimization are proposed.
- **Appendix A:** Installation Manual. An appendix provides step-by-step instructions for setting up the technical environment and dependencies required to replicate the experimental work.

2

Background

This chapter introduces basic concepts needed to fully understand this work. The former span two fields: quantum computing and artificial intelligence. Due to the breadth of these fields, we will focus only on the aspects used in this final degree thesis.

2.1 Quantum Computing

Quantum computing represents a paradigm shift from classical computation, harnessing principles of quantum mechanics to process information in fundamentally novel ways [1]. Unlike classical bits (which exist in definite states of 0 or 1), quantum computers utilize Quantum Bits (qubits) that exploit two core phenomena:

- **Superposition:** Qubits can exist in a linear combination of states, $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ where $|\alpha|^2 + |\beta|^2 = 1$, where α and β are the probability amplitudes of obtaining the state $|0\rangle$ and $|1\rangle$, respectively. This enable parallel computation on multiple states simultaneously [42]. This allows n qubits to represent 2^n states concurrently, providing exponential computational scaling [40].
- **Entanglement:** Qubits can become quantum-mechanically correlated, where the state of one qubit instantaneously determines another's, regardless of physical separation [42]. This enables quantum parallelism and coordinated operations across the entire quantum register [28].

These properties allow quantum computers to solve certain classes of problems (e.g., factorization, quantum simulation, etc.) exponentially faster than classical counterparts [40]. Two primary paradigms implement quantum computation:

1. **Adiabatic Quantum Computing (AQC):** Solves optimization problems by gradually evolving a system's Hamiltonian from an initial state to a final state encoding the solution [42]. This approach typically uses an Ising model Hamiltonian ($H = - \sum J_{ij} \sigma_i \sigma_j - \sum h_i \sigma_i$) where $\sigma_i \in \{-1, +1\}$ represent classical spin states, J_{ij} denotes interaction strengths, and h_i represents local fields. This mapping allows optimization problems to be solved through energy minimization. This approach is particularly effective for combinatorial optimization problems[1].
2. **Gate-Based Quantum Computing (GQC):** Employs sequences of quantum logic gates (unitary operations) to manipulate qubits, analogous to classical logic circuits but enriched by quantum effects [42]. This paradigm supports universal quantum computation through gate sets and enables complex algorithms such as Shor's factorization and Grover's search [1].

In this project, we focus specifically on the GQC due to its versatility in algorithm design, error correction implementations, and universal computation capabilities [28]. Our work develops quantum circuits using fundamental gates (Pauli- X , Hadamard, CNOT) to construct and test quantum algorithms,

2.2 Gate-base Quantum Computing

Gate-based quantum computing is a model in which quantum algorithms are expressed as sequences of *quantum gates*. Nevertheless, qubits can exist in superpositions of the computational basis states $|0\rangle$ and $|1\rangle$; this difference from the classical bits makes qubits really useful in the right algorithms. The state of a qubit is usually represented by a vector $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where $\alpha, \beta \in \mathbb{C}$ are the probability amplitudes of the states $|0\rangle$ and $|1\rangle$. If we measure a qubit in this state, the values $|\alpha|^2$ and $|\beta|^2$ give the probabilities of observing state 0 or 1, respectively, and since these are the only two states that can be observed (the only two states into which ψ can collapse when observed) they must fulfill the following condition $|\alpha|^2 + |\beta|^2 = 1$.

The Bloch sphere is a geometric representation of the state of a single qubit. On the Bloch sphere: the north pole corresponds to $|0\rangle$, the south pole to $|1\rangle$ and the equator includes superpositions like $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. This representation helps visualize how quantum gates rotate qubit states.

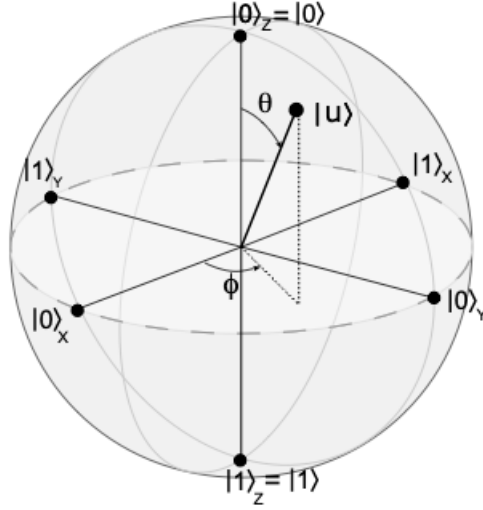


Figure 1: Bloch Sphere. Image adapted from [37].

Quantum gates are represented as unitary matrices acting on the state vectors of qubits. The quantum state of n qubits is represented by the tensor product of n Hilbert spaces, each corresponding to a single qubit. The set of gates: Hadamard gate (H), rotation gates ($R_x(\theta)$ and $R_z(\theta)$) and the CNOT gate; forms a *universal gate set*. This means any quantum computation can be decomposed into a sequence of these gates. We will now proceed to explain these gates.

2.2.1 Hadamard Gate (H)

The Hadamard gate is one of the most fundamental and widely used single-qubit gates. It creates an equal superposition of $|0\rangle$ and $|1\rangle$:

$$H |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H |1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Matrix form:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Geometrically, on the Bloch sphere, the Hadamard gate corresponds to a rotation of π radians about the axis $\frac{1}{\sqrt{2}}(\hat{x} + \hat{z})$. It swaps the x and z axes and inverts the y axis.

2.2.2 Rotation Gates

$R_x(\theta)$: X-axis Rotation: The R_x gate rotates the qubit state by an angle θ around the x -axis of the Bloch sphere:

$$R_x(\theta) = e^{-i\frac{\theta}{2}X} = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$

$R_z(\theta)$: Z-axis Rotation: The R_z gate rotates the qubit state by an angle θ around the z -axis of the Bloch sphere:

$$R_z(\theta) = e^{-i\frac{\theta}{2}Z} = \begin{bmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{bmatrix}$$

This gate changes the relative phase between $|0\rangle$ and $|1\rangle$.

2.2.3 Controlled-NOT (CNOT) Gate

The CNOT gate is a two-qubit gate that flips the state of the *target* qubit if and only if the *control* qubit is in the state $|1\rangle$:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

It introduces entanglement between qubits and is essential for building complex quantum circuits. Although often represented with distinct control and target qubits, their roles can be swapped using Hadamard transformations.

2.2.4 Pauli matrices

Although Pauli gates are not strictly necessary since they can be expressed using the previously described gates, in quantum computing, the *Pauli matrices* are fundamental single-qubit operators. They are used to represent quantum gates, describe spin systems, and appear frequently in Hamiltonians used for quantum algorithms such as QAOA and VQE. The Pauli matrices are:

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Each Pauli matrix corresponds to a rotation (up to a global phase) around one of the three axes of the Bloch sphere:

- σ_x : bit-flip or NOT gate (rotation around x -axis),
- σ_y : combined bit- and phase-flip (rotation around y -axis),
- σ_z : phase-flip (rotation around z -axis).

They satisfy the following properties:

- Hermitian: $\sigma_i^\dagger = \sigma_i$,
- Unitary: $\sigma_i^\dagger \sigma_i = I$,
- Involution: $\sigma_i^2 = I$,
- Commutation: $[\sigma_i, \sigma_j] = 2i\epsilon_{ijk}\sigma_k$,
- Anticommutation: $\{\sigma_i, \sigma_j\} = 2\delta_{ij}I$,

where $i, j, k \in \{x, y, z\}$ and ϵ_{ijk} is the Levi-Civita symbol.

In addition to their standalone use, tensor products of Pauli matrices are employed to construct Hamiltonians for multi-qubit systems. For example, the transverse-field Ising model uses a sum of tensor products involving σ_z and σ_x .

2.3 Quadratic Unconstrained Binary Optimization Problems

Building upon the foundation of gate-based quantum computing introduced in the previous Section, this section delves into QUBO problems. QUBOs represent a fundamental framework for expressing combinatorial optimization tasks using binary variables and quadratic objective functions. We will first define the QUBO formalism and its mathematical structure, explore its key properties, and highlight its diverse applications and extensions. Crucially, we will examine its specific relevance for quantum computing, including its role as a target for algorithms like QAOA discussed previously, and illustrate its practical use through the MaxCut problem.

2.3.1 Definition and Formulation

Quadratic Unconstrained Binary Optimization (QUBO) problems form a fundamental class of NP-complete optimization problems characterized by binary decision variables $x = (x_1, \dots, x_n) \in \{0, 1\}^n$ (where n is the size of the QUBO (i.e. number of variables)) and a square symmetric matrix $Q \in \mathbb{R}^{n \times n}$. The cost function is given by the quadratic form:

$$C(x) = x^T Q x = \sum_{i,j=1}^n Q_{ij} x_i x_j \quad (\text{being } x^T \text{ the transpose matrix}) \quad (1)$$

with the optimization objective being the minimization $x^* = \arg \min_{x \in \{0,1\}^n} C(x)$. Notably, any maximization problem can be converted to this form through sign inversion of Q .

2.3.2 Key Properties

QUBO problems are distinguished by their unconstrained nature, lacking any restrictions on the variables x beyond the binary domain. Their NP-completeness was established by Lucas [30], who demonstrated that all of Karp's 21 NP-complete problems admit polynomial-time mappings to QUBO formulations. Furthermore, Lodewijks [29] established a bijection between QUBO problems and Ising models through the transformation $s_i = 2x_i - 1$, where $s \in \{-1, 1\}$, while correcting and extending earlier mapping approaches.

2.3.3 Applications and Extensions

The QUBO framework encompasses numerous important optimization problems including MaxCut, Graph Coloring [38], Number Partitioning, and Quadratic Knapsack [17]. This formulation can be extended to higher-order interactions through Polynomial Unconstrained Binary Optimization (PUBO), where cubic terms like $x_i x_j x_k$ can be reduced to quadratic form via ancillary variables (e.g., introducing $x' = x_i x_j$). The Lechner-Hauke-Zoller (LHZ) model [26, 27, 25] provides alternative formulations using a lattice gauge framework with four-body interactions.

2.3.4 Applications in Quantum Computing

The QUBO formalism exhibits particular suitability for quantum computing paradigms. Its direct correspondence with Ising models makes it naturally compatible with quantum an-

nealing approaches [12]. Moreover, the binary partition structure maps naturally to qubit representations, while the quadratic objective function aligns well with the interaction terms implementable on quantum hardware. Quantum approaches like the QAOA leverage this natural mapping, offering a promising framework for exploring potential quantum advantages in combinatorial optimization. The problem's well-understood classical approximation bounds provide clear metrics for evaluating quantum algorithm performance.

2.3.5 MaxCut Problem

The MaxCut problem symbolizes a central optimization challenge in graph theory, where for a given undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with vertex set $\mathcal{V}$ and edge set $\mathcal{E}$, we seek to partition the vertices into two complementary subsets that maximize the sum of weights of crossing edges. Each edge $(i, j) \in \mathcal{E}$ has a related weight $w_{ij} = w_{ji} > 0$, with the special case of uniform weights $w_{ij} = 1$ defining the standard MaxCut problem.

Mathematical Formulation

The partition can be represented using binary variables $x = (x_1, \dots, x_{|\mathcal{V}|})$ where $x_i \in \{0, 1\}$ indicates the subset assignment of vertex i . The objective function to maximize takes the quadratic form:

$$C(\mathbf{x}) = \sum_{(i,j) \in \mathcal{E}} w_{ij} x_i (1 - x_j) \quad (2)$$

This formulation counts the weighted edges between partitions, as the term $x_i(1 - x_j)$ evaluates to 1 precisely when vertices i and j reside in different subsets. The symmetry $w_{ij} = w_{ji}$ ensures consistent counting of each edge.

Computational Complexity

MaxCut is classified as NP-hard, thus, is a problem to which every problem in NP can be reduced in polynomial time. The approximation complexity is similarly constrained, with theoretical results establishing that achieving an approximation ratio better than $16/17 \approx 0.9412$ remains NP-hard. The best-known classical approximation, the Goemans-Williamson algorithm [18], achieves a ratio of approximately 0.878 under the random hyperplane rounding

technique, though this performance guarantee relies on solving a semidefinite programming relaxation.

2.4 Quantum Approximate Optimization Algorithm

Leveraging the QUBO framework established in the previous section, this section introduces the QAOA, a hybrid approach designed to solve combinatorial optimization problems on gate-based quantum computers.

2.4.1 Variational Quantum Algorithms

The QAOA falls under the umbrella of Variational Quantum Algorithms (VQAs) [5, 41], designed to solve optimization problems on Noisy Intermediate-Scale Quantum (NISQ) devices. Noisy intermediate-scale quantum refers to the current era of quantum computing hardware, characterized by devices with 50–100 qubits that lack full error correction. These qubits are highly susceptible to noise (decoherence, gate errors), limiting circuit depth and practical scalability, but they enable early experimentation with hybrid quantum-classical algorithms. These VQAs employ the **variational principle** to minimize the expectation value of a Hamiltonian $\hat{H}$, approximating the ground state energy E_0 of a quantum system:

$$E_0 \leq \frac{\langle \psi | \hat{H} | \psi \rangle}{\langle \psi | \psi \rangle}.$$

The process involves iteratively optimizing a parameterized *ansatz* (trial wave function $|\psi\rangle$) to converge toward the lowest eigenvalue of $\hat{H}$.

The selection of an appropriate *ansatz* involves two primary strategies. Problem-inspired *ansatzes* are tailored to specific systems, such as quantum chemistry models for helium atoms [10]. Alternatively, hardware-efficient *ansatzes* utilize native quantum gates to minimize circuit depth while maintaining versatility across problems [23]. Implementation on quantum computers leverages qubits to encode wave functions, which are adjusted through parameterized quantum gates like Bloch sphere rotations. Subsequent measurements yield expectation values that enable iterative optimization. The framework finds direct application in the Variational Quantum Eigensolver (VQE), particularly in quantum chemistry contexts [39, 14, 2, 11].

The flexibility in ansatz design enables adaptation to both hardware constraints and problem-specific requirements. Furthermore, the efficient encoding of Hamiltonians as sums of unitary operations significantly facilitates quantum computation implementations.

2.4.2 Overview of QAOA

In this section, we will do an introduction on the QAOA based in [8]. The QAOA is a quantum gate model algorithm to solve combinatorial optimization problems [13]. Contrasting quantum annealing solvers [31], QAOA eliminates empirical characterization requirements. Notably, quantum annealing performance diminishes with longer execution times [9], but QAOA_{*p*} (QAOA with depth *p*) performance increases monotonically with *p*. In the asymptotic limit (*p* → ∞), QAOA_{*p*}'s approximation ratio *r* converges to 1 [43]. QAOA's structural simplicity also makes it particularly suitable for NISQ devices [34].

This section outlines QAOA—leveraging these advantages—and establishes definitions based on Farhi et al. [13] and subsequent works [21, 43, 20, 19].

Definition 1 (Objective Function) *For combinatorial optimization problems over *n*-bit binary strings $x \in \{0, 1\}^n$, the objective function f maps each configuration to a real value:*

$$f(x) : \{0, 1\}^n \rightarrow \mathbb{R}.$$

Definition 2 (Phase Hamiltonian) *The objective function f is encoded in a diagonal operator H_P acting on an *n*-qubit Hilbert space ($\dim 2^n$) satisfying the following equality:*

$$H_P|x\rangle = f(x)|x\rangle. \quad (3)$$

Consequently, optimizing f reduces to finding extremal eigenvalues of H_P .

Definition 3 (Phase Operator) *The parameterized unitary operator associated with H_P is defined as:*

$$U_P(\gamma) = e^{-i\gamma H_P}, \quad (4)$$

where $\gamma \in \mathbb{R}$ is an adjustable parameter.

Definition 4 (Mixing Operators) *The mixing Hamiltonian H_M is constructed as the sum of Pauli-*X* operators across all *n* qubits:*

$$H_M = \sum_{j=1}^n \sigma_j^x, \quad (5)$$

where σ_j^x denotes the Pauli- X operator (defined in 2.1.5) acting on qubit j .

Definition 5 (Mixing Operator) The parameterized unitary operator derived from H_M is defined as:

$$U_M(\beta) = e^{-i\beta H_M},$$

where $\beta \in \mathbb{R}$ is an adjustable parameter. In some literature, this operator is also denoted as the cost Hamiltonian $\hat{H}_C$.

Definition 6 (Initial State) The algorithm begins in a uniform superposition state across all n -qubit computational basis vectors, as dictated by quantum superposition principles:

$$|s\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle. \quad (6)$$

Using the operators defined previously, the p -level QAOA state is generated through alternating applications of phase and mixing unitaries:

$$|\psi_{\gamma,\beta}\rangle = U_M(\beta_p)U_P(\gamma_p) \cdots U_M(\beta_1)U_P(\gamma_1)|s\rangle, \quad (7)$$

where $p \geq 1$ denotes the circuit depth, and $\gamma = \gamma_1 \cdots \gamma_p$ and $\beta = \beta_1 \cdots \beta_p$ represent $2p$ adjustable parameters [13].

Repeated computational-basis measurements of state (7) yield the expectation value:

$$\langle H_P \rangle := \langle \psi_{\gamma,\beta} | H_P | \psi_{\gamma,\beta} \rangle = \langle f \rangle_{\gamma,\beta}, \quad (8)$$

equating to the objective function's expectation (Definition 1) [21].

As illustrated in Figure 2, finding the extremes $\langle H_P \rangle$ requires iterative tuning of parameters γ , β using classical optimization techniques. This parameter optimization framework establishes QAOA as a hybrid quantum-classical algorithm: Quantum hardware prepares and measures states, while classical systems optimize parameters. [34, 19, 36, 35].

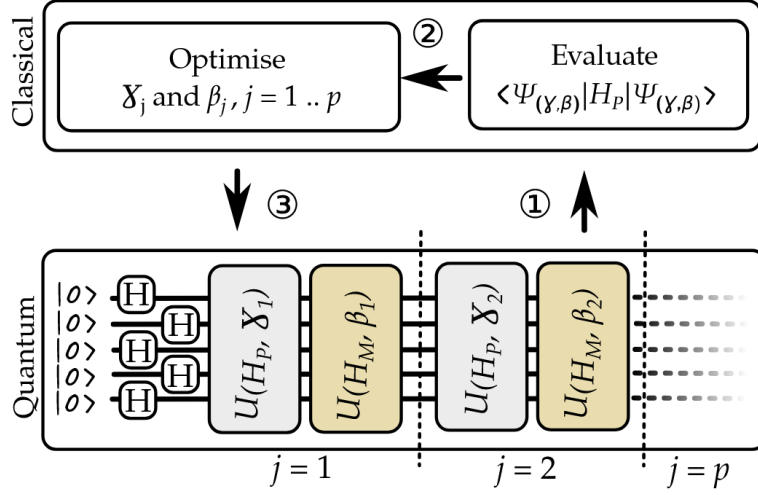


Figure 2: QAOA framework: where $U(H_p, \gamma_p) = U_P(\gamma_P)$ and $U(H_m, \beta_P) = U_m(\beta_P)$ [7]

2.4.3 QAOA Framework

After this overview, we will describe the procedure of executing the algorithm to solve a Max-Cut problem, following the exposition on [4]. In this section, we will denote the phase Hamiltonian H_P as the cost Hamiltonian $\hat{H}_C$. The original QAOA consists of the following steps:

1. **Problem Encoding:** Define a cost Hamiltonian $\hat{H}_C$ whose highest-energy eigenstate encodes the optimal solution. A non-commuting mixer Hamiltonian $\hat{H}_M$ is simultaneously defined.
2. **MaxCut Specific Formulation:** for a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, with weights w_{ij} :

$$\hat{H}_C = \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} (I - Z_i Z_j), \quad (9)$$

[4]

$$\hat{H}_M = \sum_{j \in \mathcal{V}} X_j, \quad (10)$$

where: I is the identity operator, Z_j and X_j are the Pauli operators (defined in Section 2.2.4) acting on qubit j and $\hat{H}_C$ is diagonal in the computational basis.

3. **Variable Mapping:** Binary variables $x_i \in \{0, 1\}$ map to spin operators via:

$$x_i \rightarrow \frac{1}{2}(1 - Z_i). \quad (11)$$

This establishes $\hat{H}_C$ (9) as the operator equivalent of the objective function (2).

4. **State initialization:** Initialize the circuit in the state $|s\rangle$:

$$|s\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x} \in \{0,1\}^n} |\mathbf{x}\rangle, \quad (12)$$

where n is the number of qubits and $n = |\mathcal{V}|$. The state $|s\rangle$ corresponds to the highest energy state of the Pauli-X basis, i.e., to the highest energy state of the mixer Hamiltonian $\hat{H}_M$.

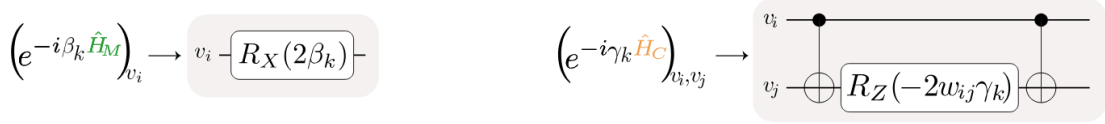


Figure 3: Quantum circuit implementation of mixer (left) and cost (right) unitary layers derived from Hamiltonians $\hat{H}_M$ and $\hat{H}_C$. **Mixer Layer (v_i -local):** The vertex-specific component of $\hat{U}_M(\beta_k)$ is defined as: $(e^{-i\beta_k \hat{H}_M})_{v_i} \equiv e^{-i\beta_k X_i} = R_{X_i}(2\beta_k)$ where β_k is the variational parameter for layer k , implementing an X -rotation on qubit i . **Cost Layer ((v_i, v_j) -local):** The edge-specific component of the cost unitary is: $(e^{-i\gamma_k \hat{H}_C})_{v_i, v_j} = R_{Z_i Z_j}(-2w_{ij}\gamma_k)$ with γ_k as the layer's variational parameter, implementing a ZZ -rotation scaled by edge weight w_{ij} . The right panel shows its decomposition [4]

5. **Quantum Circuit Construction:** Define and implement the parameterized unitary operators:

$$\hat{U}_C(\gamma) = e^{-i\gamma \hat{H}_C} = \prod_{i=1, j < i}^n R_{Z_i Z_j}(-2w_{ij}\gamma), \quad (13)$$

$$\hat{U}_M(\beta) = e^{-i\beta \hat{H}_M} = \prod_{i=1}^n R_{X_i}(2\beta), \quad (14)$$

where γ, β are variational parameters. These operators constitute:

- Cost layer ($\hat{U}_C(\gamma)$): Composed of ZZ-rotations for each edge
- Mixer layer ($\hat{U}_M(\beta)$): Composed of X-rotations for each vertex

A single QAOA layer combines one cost and one mixer layer. Multiple layers can be stacked to form deeper circuits. As depicted in Figure 3:

- (a) Mixer components implement as single-qubit R_X gates
- (b) Cost components implement as two CNOT gates with local R_Z rotation

6. **Quantum Circuit Initialization:** Define the QAOA circuit depth $p \geq 1$ and initialize $2p$ variational parameters

$$\gamma = (\gamma_1, \gamma_2, \dots, \gamma_p) \text{ with } \gamma_k \in [0, 2\pi)$$

$$\beta = (\beta_1, \beta_2, \dots, \beta_p) \text{ with } \beta_k \in [0, \pi)$$

for $k = 1, \dots, p$. The quantum circuit prepares the final state:

$$|\psi_p(\gamma, \beta)\rangle = e^{-i\beta_p \hat{H}_M} e^{-i\gamma_p \hat{H}_C} \dots e^{-i\beta_1 \hat{H}_M} e^{-i\gamma_1 \hat{H}_C} |s\rangle. \quad (15)$$

7. **Cost Function Evaluation:** Compute the expectation value of $\hat{H}_C$ for the parameterized state $|\psi_p(\gamma, \beta)\rangle$, equivalent to the algorithm's solution cost. This is estimated through repeated computational-basis measurements:

$$F_p(\gamma, \beta) = \langle \psi_p(\gamma, \beta) | \hat{H}_C | \psi_p(\gamma, \beta) \rangle \quad (16)$$

8. **Classical Optimization** Employ classical optimization techniques to iteratively refine the parameters. The optimization objective is to maximize the expectation value:

$$(\gamma^*, \beta^*) = \arg \max_{\gamma, \beta} F_p(\gamma, \beta) \quad (17)$$

where (γ^*, β^*) denotes the optimal parameter set.

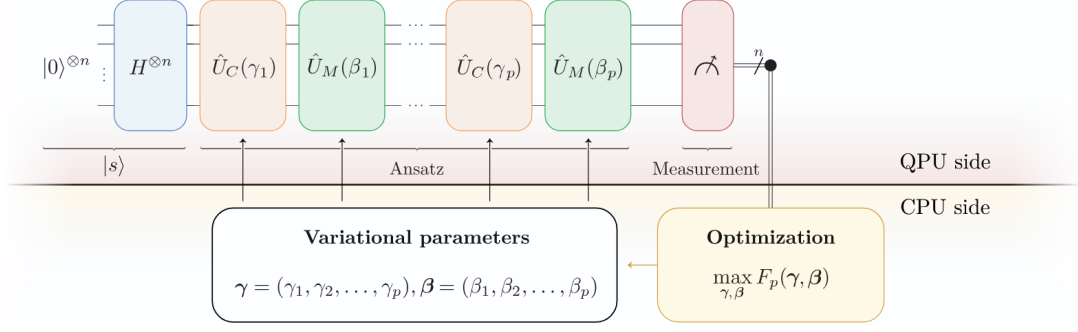


Figure 4: Diagram of the p -layer QAOA's quantum-classical workflow [4]

Figure 4 illustrates a representative QAOA circuit implementation. Following parameter optimization, the solution quality is quantified by the approximation ratio:

$$\alpha = \frac{F_p(\gamma^*, \beta^*)}{C_{\max}}, \quad (18)$$

where the "optimal state" $|\psi_p(\gamma^*, \beta^*)\rangle$ encodes the problem solution.

Notably, QAOA initialization typically employs the highest-energy eigenstate of the mixer Hamiltonian (e.g., in MaxCut implementations) rather than the ground state. This configuration still satisfies adiabatic theorem requirements. Beyond standard QAOA's digitized-adiabatic approach (Section 2.5), Barraza et al. [3] recently introduced an analog variant executable on quantum annealing hardware.

2.5 Machine Learning

Artificial Intelligence (AI) encompasses the development of systems capable of performing tasks that typically require human intelligence. As outlined in our structural overview, AI spans several key domains: problem solving (e.g. search, optimization, constraint satisfaction) knowledge, reasoning, and planning (e.g. logic systems, automated planning); uncertain knowledge and reasoning (e.g. probabilistic methods); machine learning (e.g. data-driven pattern recognition) and communicating, perceiving, and acting (e.g. NLP, computer vision, robotics).

Machine Learning serves as a fundamental subset of AI that enables systems to learn from data without explicit programming. Rather than following rigid instructions, ML algorithms

identify patterns and relationships within datasets to make predictions or decisions. This data-driven approach has become the cornerstone of modern AI applications.

The two primary paradigms of machine learning are:

1. **Supervised Learning:** Algorithms learn from *labeled training data* where each input example is paired with the correct output. The model generalizes from these examples to make predictions on new, unseen data. Common applications include: classification (e.g., spam detection, image recognition) and regression (e.g., price prediction, risk assessment).
2. **Unsupervised Learning:** Algorithms analyze *unlabeled data* to discover inherent structures or patterns without predefined outcomes. Key applications include: clustering (e.g., customer segmentation, anomaly detection) and dimensionality reduction (e.g., data visualization, feature extraction).

Given its versatility and proven effectiveness in predictive tasks, supervised learning will serve as the primary machine learning methodology employed throughout this thesis. While other approaches like reinforcement learning and deep learning extend these foundations, supervised learning provides the structured framework necessary for our research objectives. Its reliance on labeled datasets will enable precise modeling of the relationships between input features and target variables in our experimental domain.

2.5.1 Supervised Machine Learning

In supervised learning, a dataset consists of N labeled examples denoted $\{(x_i, y_i)\}_{i=1}^N$. Each x_i is termed a feature vector. This vector contains D dimensions ($j = 1, \dots, D$), where each component $x^{(j)}$ represents a feature describing a specific characteristic of the example. Feature consistency is maintained across the dataset: if position j in a feature vector represents a specific characteristic (e.g., weight) for one example, it represents the same characteristic for all examples. The label y_i may belong to a finite set of classes $\{1, 2, \dots, C\}$, represent a real number, or exhibit a more complex structure (e.g., vector, matrix). Within this text, y_i is restricted to either categorical classes or real values. Classes denote categories; for example, in email spam detection, the classes would be spam, not _spam.

The objective of a supervised learning algorithm is to utilize the dataset to generate a model. This model accepts a feature vector x as input and produces output enabling the deduction of the corresponding label. For example, a model trained on medical data could accept a feature vector describing an individual and output an estimated probability of cancer diagnosis.

2.5.2 Neural Networks

One of the supervised learning technique is neural networks represent mathematical functions $y = f_{NN}(\mathbf{x})$ constructed through nested function composition. A 3-layer network producing scalar output exemplifies this structure: $y = f_{NN}(\mathbf{x}) = f_3(f_2(f_1(\mathbf{x})))$. Each layer function f_l follows the standardized formulation $f_l(\mathbf{z}) \stackrel{\text{def}}{=} g_l(\mathbf{W}_l \mathbf{z} + \mathbf{b}_l)$, where l denotes the layer index. The activation function g_l constitutes a predefined nonlinear transformation selected prior to model training.

The weight matrices $\mathbf{W}_l$ and bias vectors $\mathbf{b}_l$ comprise learnable parameters optimized via gradient descent. Optimization targets task-specific cost functions, such as Mean Squared Error for regression. This parameter update mechanism directly extends the gradient descent approach used in logistic regression, with sigmoid activation representing a special case of g_l .

The matrix formulation $\mathbf{W}_l$ originates from the vector-valued nature of layer computations. Each row $\mathbf{w}_{l,u}$ of $\mathbf{W}_l$ corresponds to a computational unit, generating pre-activation values $a_{l,u} = \mathbf{w}_{l,u} \mathbf{z} + b_{l,u}$. The layer output is computed as $f_l(\mathbf{z}) = [g_l(a_{l,1}), g_l(a_{l,2}), \dots, g_l(a_{l, \text{size}_l})]$, where size_l indicates the layer's unit count and g_l operates as an element-wise scalar function.

The final layer function f_3 adapts to problem requirements: scalar output for regression tasks, or vector output for multi-dimensional predictions. This architectural flexibility enables neural networks to address diverse machine learning problems through consistent mathematical foundations.

The multilayer perceptron (often termed “vanilla neural network”) exemplifies this architecture. Its nested nonlinear transformations enable complex function approximation capabilities beyond simpler models like linear regression. The chained composition of differentiable functions permits gradient-based optimization through backpropagation, where errors propagate backward through layers to update weight parameters.

2.5.3 Graph Neural Networks

Graph Neural Networks (GNNs) represent a specialized class of neural functions $\mathbf{Y} = f_{GNN}(\mathcal{G})$ designed to operate on graph-structured data. Unlike standard neural networks that process vector inputs, GNNs handle graph-structured inputs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V}$ is the set of nodes and $\mathcal{E}$ the set of edges. A fundamental GNN layer implements *message passing*:

$$\mathbf{h}_v^{(k)} = \phi^{(k)} \left(\mathbf{h}_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}(\mathbf{h}_v^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{e}_{vu}) \right)$$

where:

- $\mathbf{h}_v^{(k)} \in \mathbb{R}^d$: Feature vector of node v at layer k (with $\mathbf{h}_v^{(0)} = \mathbf{x}_v$)
- $\mathcal{N}(v)$: Neighborhood of node v
- $\phi^{(k)}, \psi^{(k)}$: Learnable transformation functions
- $\bigoplus$: Permutation-invariant aggregation operator (e.g., sum, mean, max)
- $\mathbf{e}_{vu}$: Optional edge features

Each GNN layer $f^{(k)}$ updates node representations by combining local node features with aggregated neighborhood information:

$$f^{(k)}(\mathbf{H}^{(k-1)}) = \sigma(\mathbf{\hat{A}}\mathbf{H}^{(k-1)}\mathbf{W}^{(k)})$$

- $\mathbf{H}^{(k)} \in \mathbb{R}^{|\mathcal{V}| \times d_k}$: Node embedding matrix at layer k
- $\mathbf{\hat{A}}$: Normalized adjacency matrix with self-connections
- $\mathbf{W}^{(k)} \in \mathbb{R}^{d_{k-1} \times d_k}$: Learnable weight matrix
- σ : Element-wise activation function (e.g., ReLU)

The final graph representation $\mathbf{y} = f_{GNN}(\mathcal{G})$ is obtained through hierarchical composition:

$$\mathbf{H}^{(K)} = f^{(K)} \circ f^{(K-1)} \circ \dots \circ f^{(1)}(\mathbf{X})$$

GNN parameters $\theta = \{\mathbf{W}^{(k)}, \mathbf{a}\}$ are optimized via gradient descent on task-specific objectives:

- **Node-level:** Cross-entropy for classification $\mathcal{L} = -\sum_{v \in \mathcal{V}_L} \mathbf{y}_v \log(\hat{\mathbf{y}}_v)$
- **Graph-level:** Graph pooling with $\mathbf{y}_G = \text{READOUT}(\{\mathbf{h}_v^{(K)} | v \in \mathcal{V}\})$

The architecture’s inductive bias enables state-of-the-art performance on relational data tasks including:

- Molecular property prediction (graph regression)
- Social network analysis (node classification)
- Recommendation systems (link prediction)
- Knowledge graph completion (relation prediction)

This structural generalization beyond grid-like data distinguishes GNNs from convolutional networks, while preserving the differentiable programming paradigm that enables end-to-end learning.

2.5.4 Gradient Boosting

Gradient boosting constructs predictive models through sequential integration of weak learners, typically decision trees. This iterative approach minimizes differentiable loss functions for both regression and classification tasks. The method exhibits fundamental connections to gradient descent optimization, with each new learner correcting errors from the current ensemble.

The regression algorithm commences with initialization: $f_0(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N y_i$. For each iteration $m = 1$ to M , three steps occur: first, residuals $\hat{y}_i = y_i - f_{m-1}(\mathbf{x}_i)$ are computed; second, a new tree f_m is trained on $\{(\mathbf{x}_i, \hat{y}_i)\}_{i=1}^N$; third, the model updates via $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \alpha f_m(\mathbf{x})$. The final model combines all components: $f(\mathbf{x}) = f_0(\mathbf{x}) + \alpha \sum_{k=1}^M f_k(\mathbf{x})$.

Classification begins with log-odds initialization: $f_0 = \ln\left(\frac{p}{1-p}\right)$ where $p = \frac{1}{N} \sum y_i$. Each iteration $m = 1$ to M executes four steps: Gradient computation $g_i = \frac{\partial L_f}{\partial f} = \frac{1}{e^{f(\mathbf{x}_i)} + 1}$ precedes training tree f_m on $\{(\mathbf{x}_i, g_i)\}_{i=1}^N$. Optimal step size determination $\rho_m = \arg \max_{\rho} L_{f+\rho f_m}$ is followed by model update $f \leftarrow f + \alpha \rho_m f_m$. Final probabilities derive from $\Pr(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-f(\mathbf{x})}}$.

The method optimizes mean squared error for regression and log-likelihood for classification. Its primary effect is bias reduction, contrasting with bagging's variance reduction. Overfitting is mitigated through tree depth constraints and learning rate modulation. The sequential training architecture inherently limits parallelization capabilities.

Three critical parameters require tuning: The number of trees (M) governs model complexity, the learning rate (α) scales tree contributions, and tree depth regulates weak learner complexity. These collectively determine model accuracy, training duration, and generalization capacity.

Gradient boosting delivers state-of-the-art predictive accuracy and effectively handles high-dimensional data. Its robustness extends to outliers and missing values, with flexibility across loss functions. Limitations include sequential training constraints, hyperparameter sensitivity, overfitting propensity relative to bagging methods, and computational intensity for large datasets.

Proposal Overview

This proposal addresses a fundamental challenge in variational quantum algorithms, and the QAOA in particular, which is the parameter optimization. The goal was to present a novel framework combining quantum computing with machine learning techniques to significantly reduce the computational and cost overhead associated with conventional parameter optimization methods. By leveraging supervised learning approaches, this project aimed to develop an efficient parameter prediction model that maintains solution quality while decreasing quantum resource requirements.

The Quantum approximate optimization algorithm has emerged as a promising approach for solving combinatorial optimization problems on near-term quantum devices. However, its practical implementation faces significant challenges:

- **Parameter optimization overhead:** Each QAOA layer requires optimization of its parameters (γ, β) , with $2p$ parameters needed for depth- p circuits.
- **Exponential resource scaling:** Parameter optimization requires numerous quantum circuit evaluations.

Traditional optimization approaches (e.g., BFGS, COBYLA, Nelder-Mead) become prohibitively expensive as problem size increases. This research addresses these limitations through a machine learning framework that learns optimal parameter landscapes across problem instances.

The central objective of this project was to develop a **machine learning-driven parameter optimization framework** for QAOA that substantially reduces quantum resource requirements while maintaining solution quality.

This objective was addressed through the integration of quantum computing and machine learning paradigms, with a specific focus on:

- Reducing quantum circuit evaluation counts by leveraging predictive models to minimize expensive quantum processor calls during parameter optimization
- Maintaining solution quality metrics at levels comparable to traditional optimization approaches
- Ensuring generalization across problem instances through domain-adaptive architectures that transfer learning between different graph structures and problem sizes
- Developing efficient classical ML surrogate models - machine learning approximations trained to emulate QAOA's parameter-to-output relationship, replacing quantum circuit executions with faster classical predictions while preserving fidelity

The initial phase was focused on generating a robust benchmark dataset through the following procedure:

1. We have produced combinatorial optimization problems. These incorporated diverse Erdős-Rényi graphs parameterized by graph density and varied weight distributions.
2. Subsequently, classical optimization techniques generated optimal parameters.

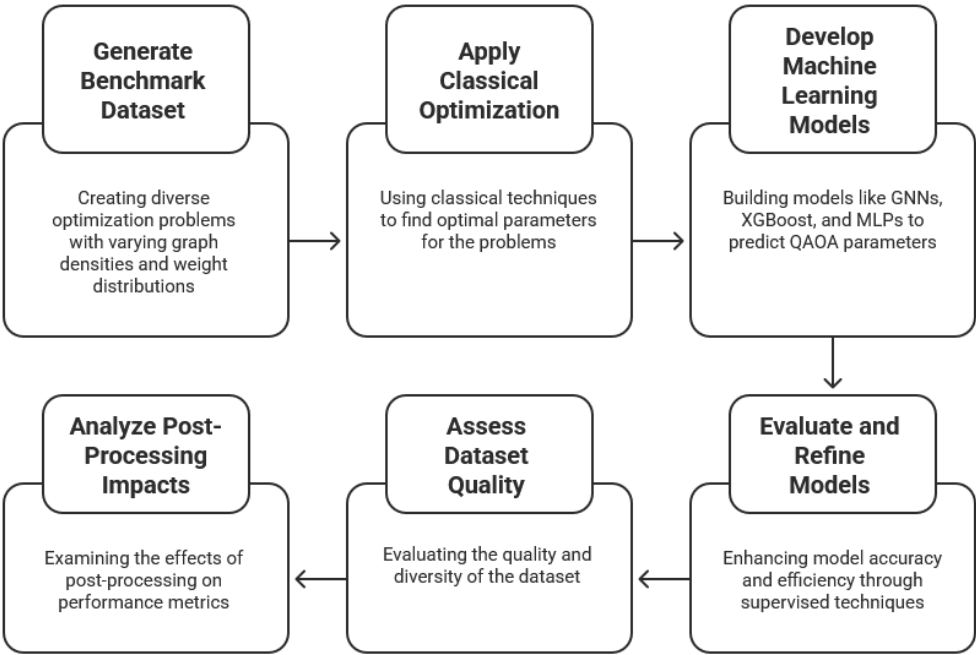
The second phase centers on developing machine learning models to predict optimal QAOA parameters. Our approach will explore multiple model architectures:

- Graph neural networks
- XGBoost gradient-boosted trees
- Neural networks (MultiLayer Perceptron)

The concluding phase implemented comprehensive model evaluation and refinement protocols. This encompasses strategic enhancements to improve model accuracy and efficiency through supervised techniques, selection of relevant instance features, and refinement of learning task modeling. The process included incorporating improved problem representations, optimizing parameters, and adjusting key configurations.

Additional evaluation criteria focus on assessing dataset quality and diversity, along with analyzing post-processing impacts on performance metrics. The pipeline can be illustrated in the following diagram.

Research Methodology for QAOA Parameter Prediction



4

Experimental Design

Building upon the theoretical foundation established in previous sections, we now present the systematic implementation of our quantum machine learning framework. The following subsections provide comprehensive technical details for each operational phase, maintaining rigorous reproducibility standards while highlighting phase-specific innovations:

- **Section 4.1 (Dataset construction)** details the dataset generation process, including quantum problem instance creation, graph feature extraction, and preprocessing pipelines tailored for QAOA parameter prediction.
- **Section 4.2 (XGBoost Implementation)** presents the gradient-boosted tree model for initial parameter estimation, with emphasis on feature importance analysis and Bayesian hyperparameter optimization.
- **Section 4.3 (Multilayer Perceptron Deployment)** documents the deep neural network architecture, specifying layer configurations, activation functions, and regularization strategies for high-dimensional optimization landscapes.
- **Section 4.4 (Graph Neural Network Integration)** develops the graph-to-parameter translation framework, explaining message-passing mechanisms and graph pooling operations adapted for quantum circuit topologies.
- **Section 4.5 (Circuit-Level Data Processing)** addresses hardware-aware feature engineering, covering noise modeling and calibration data incorporation for real-device deployment.

Each phase maintains consistent evaluation metrics—particularly approximation ratio preservation and quantum resource reduction—to enable direct comparison across methodologies. This phased implementation mirrors our framework’s conceptual workflow while providing complete technical specifications for replication.

4.1 Dataset

The data generated is composed of 250 Erdős-Rényi graphs for MaxCut problems. The 250 graphs have the following properties:

- They have 10, 12, 15, 20 and 25 nodes.
- We have used 0.5, 0.6, 0.7, 0.8 and 0.9 as probabilities of edge creation, which represents the probability ruling whether an edge will be created or not between two vertices.

We have generated 10 graphs per combination of the number of nodes and the probability of edge creation. This provides a relatively large and diverse set of graphs, ensuring our models can generalize appropriately.

We then used the dataset of the 250 graphs to generate the data on which the models will be trained and evaluated. The methodology consists of four main components [22], which are explained in the following subsections.

4.1.1 QUBO Formulation for MaxCut

Given a graph $G = (V, E)$ with edge weights w_{ij} , we construct the QUBO matrix Q as follows:

$$Q_{ii} = - \sum_{j \in N(i)} w_{ij}, \quad Q_{ij} = 2w_{ij} \text{ for } i \neq j \quad (19)$$

where $N(i)$ denotes the neighbors of node i . The matrix Q is initialized as zero and populated by iterating through all edges.

4.1.2 Pauli Hamiltonian Conversion

The QUBO matrix is transformed into a Pauli-Z Hamiltonian suitable for QAOA:

$$H = \sum_{i < j} Q_{ij} Z_i Z_j \quad (20)$$

The conversion process involves:

- Generating Pauli strings with Z operators at positions corresponding to non-zero Q_{ij} terms
- Preserving the coefficients from the QUBO matrix
- Using identity operators (I) for all other qubit positions

4.1.3 QAOA Circuit Preparation and Execution

We implemented a QAOA ansatz with $p = 2$ layers (due to simulation capacities we could not go beyond $p=2$), alternating between the problem Hamiltonian H (phase separation) and a mixer Hamiltonian (state exploration). The quantum circuit was transpiled for optimal execution on Qiskit's `qasm_simulator` backend. Key aspects included:

- **Parameter Initialization:** The initial angles γ and β were set to π and $\pi/2$, respectively, based on empirical heuristics.
- **Measurement:** All qubits were measured in the computational basis to evaluate the expectation value of H .

4.1.4 Hybrid Quantum-Classical Optimization

The QAOA parameters were optimized classically using a gradient-free method:

- **Cost Function:** The expectation value of H was estimated via 1,000 shots per circuit evaluation (as in the equation 8).
- **Minimization:** The COBYLA optimizer adjusted parameters to minimize the energy, with convergence tolerance $\text{tol} = 10^{-2}$. Intermediate cost values were tracked to monitor progress.

4.1.5 Dataset Curation for Empirical Analysis

To enable scalable analysis, we systematically compiled a dataset linking graph properties to optimized QAOA parameters:

- **Features:** Since the weight of the edge is unique ($w_{ij} = w_{ji}$), for each graph, only non-zeros uppertriangular the upper triangular entries of Q were extracted as features, due to Q being a symmetric matrix, i.e. $Q_{ij} = Q_{ji}$. Thus, the amount data stored was reduced without compromising data quality.
- **Labels:** The optimized parameters $\{\gamma_1, \beta_1, \gamma_2, \beta_2\}$ for each graph were stored as targets.
- **Storage:** Data was saved in CSV files, partitioned by node count and aggregated into a unified dataset. This was done to ease their use by the ML unit. Also, facilitate their exploitation by other researchers and practitioners.

In total, 6 datasets were created, one for each number of nodes and one containing all the data. Due to the difference in size between the QUBO matrices (because of the difference in the number of nodes between the graphs) multiple types of padding were considered, using statistical measurements, zero-padding, etc. Finally, the full dataset was completed by zero-padding.

4.2 XGBoost

In this and the following sections, we explain how we modeled each machine learning technique to address this problem. We begin with the characteristics of XGBoost.

4.2.1 Model Architecture and Training Protocol

We employed the XGBoost regression framework [6] to predict optimal QAOA parameters. The model architecture was configured with the following hyperparameters:

- Objective function: `reg:squarederror` (squared error regression)
- Number of estimators: 500
- Maximum depth: 4
- Learning rate: 0.05
- Subsample ratio: 0.8

- Column subsample ratio: 0.8

The training protocol implemented an 80%-20% stratified split between training and testing datasets, with a fixed random state (`random_state=42`) for reproducibility. Model training was performed on the QUBO-derived feature matrices, with computational efficiency measurements recorded during execution.

4.2.2 Feature Engineering and Data Representation

The input feature space was constructed from the QUBO matrix elements, represented as:

$$\mathbf{X} = \{Q_{ij} \mid 1 \leq i, j \leq n\}$$

where n denotes the number of nodes in each graph instance. The target variables corresponded to the solution vectors:

$$\mathbf{y} = \{x_i \mid 1 \leq i \leq n\}$$

This representation encodes the combinatorial optimization landscape directly into the feature space, enabling the model to learn parameter mappings from the QUBO formulation.

4.2.3 Evaluation Metrics

Model performance was quantified using two complementary error metrics. This was done to assess the performances of the model from different perspectives.

1. Mean Squared Error (MSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

2. Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Training time τ was also recorded for each model instance using high-resolution timing:

$$\tau = t_{\text{end}} - t_{\text{start}} \text{ (measured in seconds)}$$

where t_{start} and t_{end} represent the timestamps before and after model, respectively. This provides a quantitative assessment of computational requirements across problem scales.

These metrics provide a comprehensive assessment of both absolute error magnitude and relative prediction accuracy.

4.2.4 Experimental Setup

The training pipeline was executed across five distinct graph size categories (10, 12, 15, 20, 25 nodes) plus a combined full dataset. For each category:

1. Dataset loaded from CSV storage
2. Feature-target separation performed
3. Train-test split implemented
4. Model instantiated and trained
5. Performance metrics calculated
6. Model serialized for future deployment

4.3 Multilayer Perceptron

Just as we described training XGBoost in the previous section, here we outline the same process for the multilayer perceptron.

4.3.1 Neural Network Architecture

We implemented a multilayer perceptron (MLP) regression model using Scikit-Learn's `MLPRegressor`. The optimized network architecture, determined through hyperparameter tuning (Section 4.3.2), consists of:

- Input layer dimensionality: n^2 features (QUBO matrix elements)
- Hidden layers: 3 fully-connected layers with (128, 64, 32) units
- Output layer: n neurons (solution vector $\mathbf{x}$)
- Activation function: Logistic sigmoid ($\sigma(z) = \frac{1}{1+e^{-z}}$)

4.3.2 Training Configuration

The model configuration resulted from extensive hyperparameter optimization using GridSearchCV:

```
activation = "logistic"
solver = "lbfgs"
max_iter = 5000
tol = 0.1
random_state = 1
```

The limited-memory BFGS (L-BFGS) optimizer was selected for its efficiency in low-dimensional parameter spaces, with convergence tolerance relaxed to accelerate training.

4.3.3 Hyperparameter Optimization

A comprehensive grid search established the optimal architecture:

Parameter	Tested Values
hidden_layer_sizes	(50), (100), (100, 50), (128, 64, 32)
activation	{'identity', 'logistic', 'tanh', 'relu'}
solver	{'lbfgs', 'sgd', 'adam'}

The final configuration (Section 4.3.2) demonstrated superior performance across validation folds ($k = 3$ cross-validation).

4.3.4 Evaluation Metrics

Model accuracy was evaluated using identical metrics to the XGBoost implementation (section 4.2.3). Training duration τ was measured with millisecond precision using Python's `time` module.

4.3.5 Experimental Execution

The training pipeline followed an identical structure to the gradient boosting implementation:

1. Dataset loading from CSV storage

2. Feature-target separation ($\mathbf{Q} \rightarrow \mathbf{x}$)
3. Stratified train-test partitioning (80% – 20% split)
4. Model instantiation with optimized parameters
5. Performance evaluation on test set
6. Model serialization via `jobjlib`

All experiments used identical graph datasets (Section 4.1) for direct comparison.

4.3.6 Computational Considerations

The L-BFGS solver provides advantages for moderately sized networks:

- **Memory efficiency:** Avoids Hessian storage ($O(n^2)$)
- **Convergence speed:** Superlinear convergence near optima
- **Batch optimization:** Full dataset utilization per iteration

The `max_iter=5000` and `tol=0.1` parameters balance convergence guarantees with computational feasibility.

4.4 Graph Neural Networks

Finally, in this section, we detail our use of Graph Neural Networks.

4.4.1 Architecture Design

We implemented a graph convolutional network (GCN) using the PyTorch Geometric framework [15]. The architecture follows established graph neural network principles [24] with the following configuration:

- **Input layer:** Node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times n}$ (identity matrix)

- **Graph convolution layers:**

$$\mathbf{H}^{(1)} = \text{ReLU} \left(\hat{\mathbf{A}} \mathbf{X} \mathbf{W}^{(0)} \right)$$

$$\mathbf{H}^{(2)} = \text{ReLU} \left(\hat{\mathbf{A}} \mathbf{H}^{(1)} \mathbf{W}^{(1)} \right)$$

where $\hat{\mathbf{A}}$ is the normalized adjacency matrix, with hidden dimension 32

- **Readout layer:** Global mean pooling
- **Output layer:** Linear transformation to n -dimensional solution vector

4.4.2 Graph Data Construction

The QUBO matrices were transformed into graph representations using the following mapping:

$$\mathbf{Q} \rightarrow \mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$$

where:

- Nodes $\mathcal{V}$ correspond to variables ($|\mathcal{V}| = n$)
- Edges $\mathcal{E}$ connect interacting variables ($e_{ij} \in \mathcal{E}$ when $Q_{ij} \neq 0$)
- Edge weights $w_{ij} = Q_{ij}$

Node features were initialized as one-hot vectors to preserve structural identity.

4.4.3 Training Protocol

The model was optimized with the following parameters:

- Optimizer: Adam ($\eta = 0.01, \lambda = 5 \times 10^{-4}$)
- Batch size: 1 (whole-graph processing)
- Epochs: 100
- Loss function: Mean squared error (MSE)
- Validation split: 80%-20% stratified partition

The implementation followed PyTorch Geometric best practices [16] for efficient message passing.

4.4.4 Evaluation Metrics

Model performance was assessed using identical metrics to the XGBoost implementation (section 4.2.3). Training time τ was recorded end-to-end, including data loading and preprocessing.

4.4.5 Computational Environment

Experiments were conducted with hardware acceleration where available:

$$\text{device} = \begin{cases} \text{"cuda"} & \text{if GPU available} \\ \text{"cpu"} & \text{otherwise} \end{cases}$$

The graph batching strategy leveraged PyG’s DataLoader for efficient memory utilization.

4.5 Circuit Level Data

In the previous sections, we showed how to build a machine learning surrogate model from the features of the graph to be partitioned (MaxCut). The main advantage of this approach is its direct applicability to any new MaxCut instance, since it incorporates all instance-specific details (edge weights). Its drawback, however, is that you either need separate models for each graph size (what if no model exists for the exact size you need?) or you must use zero-padding to equalize graph sizes, resulting in many inputs that are zeros and contribute little or even confuse the model.

To overcome these issues, a second approach has been proposed: instead of modeling the graph directly, use features of the quantum circuit that encodes the problem as inputs to our ML models. This decouples the ML input size from the graph size, but at the cost of losing some instance-specific precision by not using the actual weights directly.

The circuit-level data processing pipeline serves to extract quantum hardware characteristics that influence parameter optimization in QAOA. This functionality is implemented through three interconnected components that transform combinatorial problems into quantum-circuit descriptors suitable for machine learning.

Circuits are decomposed into a restricted gate set (RX, RZ, CX gates with preserved measurement operations) without optimization to maintain structural fidelity. This function then

calculates: circuit depth representing the longest path of sequential operations; qubit count requirements; and gate-specific counts for rotation (RX, RZ) and entanglement (CX) operations. These metrics capture the quantum resource requirements essential for understanding parameter-performance relationships.

The dataset generation pipeline orchestrates the full feature extraction workflow. Beginning with graph-based optimization problems stored in CSV format, it reconstructs symmetric QUBO matrices from flattened vector representations. For each problem instance, it constructs a parameterized QAOA circuit with specified layer depth, converts the QUBO to a Pauli Hamiltonian, and assigns optimal parameters previously determined through classical optimization. The transpiled circuit undergoes feature extraction. This process enriches the machine learning dataset by combining original problem parameters with quantum-circuit characteristics, preserving both graph properties and quantum-hardware metrics while removing redundant QUBO matrix representations.

This circuit-level feature engineering enables machine learning models to identify correlations between quantum circuit topologies and optimal parameter configurations. By capturing hardware-aware characteristics like entanglement requirements and rotational complexity, the extracted features address the challenge of generalizing parameter strategies across diverse problem instances and quantum hardware constraints. The refined models now utilize five key quantum circuit parameters as input features:

1. Circuit depth
2. Number of qubits
3. RX gate count
4. RZ gate count
5. CX gate count

An instance of an implementation of this experimental design can be found in the following link:https://github.com/jann9/TFG_QC.git.

5

Results and comparison

To rigorously evaluate the predictive capabilities and computational efficiency of the implemented machine learning architectures—Multi-Layer Perceptron (MLP), XGBoost (XGB), and Graph Neural Network (GNN)—we systematically extracted performance metrics from training logs and transformed them into an ensemble of visual comparisons. This section details how key quantitative results (RMSE, MAPE, and training time) were analyzed through purpose-built visualizations to address three critical dimensions:

- Performance comparison of different models across classical and quantum architectures
- Performance degradation due to padding when using the full dataset
- Comparative analysis between QUBO formulations and circuit-level data representations

Using Python’s Seaborn and Matplotlib libraries, two tiers of visual analytics were generated:

1. Method-Specific Dashboards: Individual plots isolating performance trends within each architecture (e.g., how RMSE evolves with node size in MLPs).
2. Cross-Architecture Comparisons: Integrated visualizations contrasting all three models to identify dominance patterns and opportunity costs.

5.1 XGBoost

The XGBoost models were evaluated with the metrics previously mentioned (4.2.3) in both the QUBO datasets and the circuit level data.

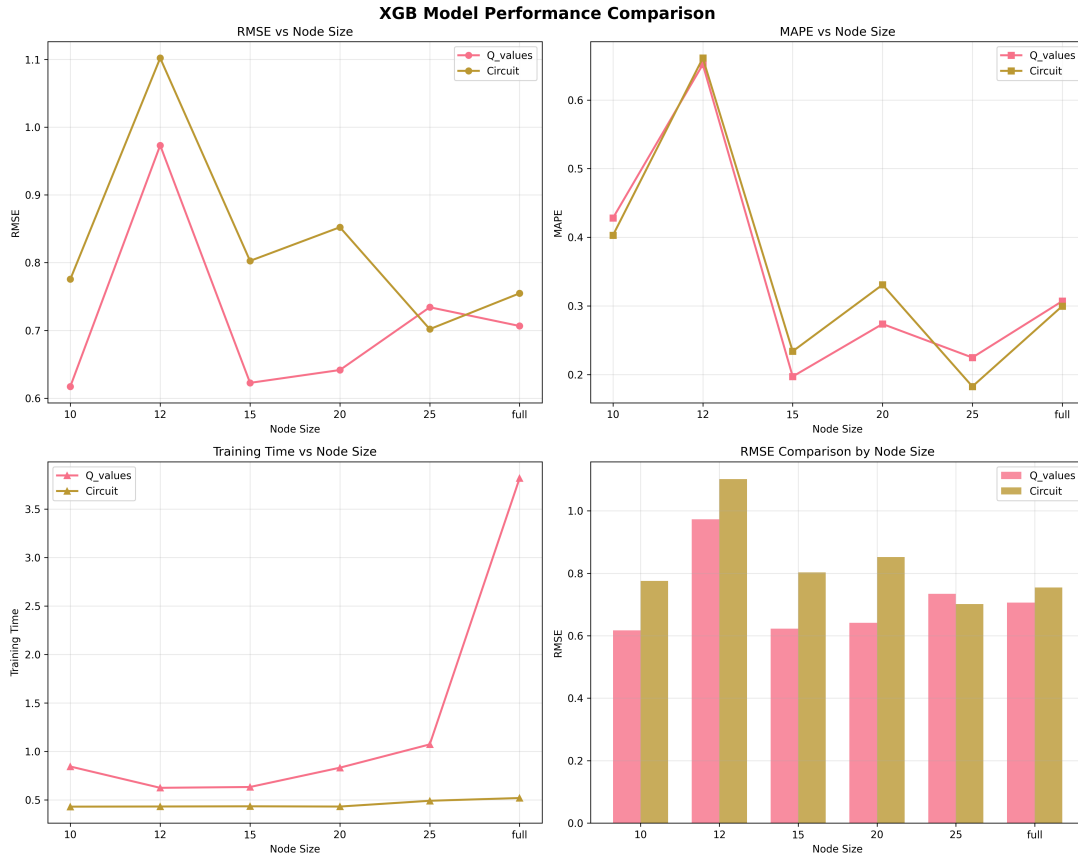


Figure 5: XGB Model Performance

The outcome of this evaluation is presented in Figure 5:

- **RMSE and MAPE:** Model performance generally improves with larger dataset sizes, as evidenced by decreasing error rates. However, the 12-node dataset represents a notable exception where prediction accuracy significantly deteriorates, indicating exceptional modeling challenges specific to this configuration. Models developed with circuit level data performed worse than those who used QUBO matrix throughout all node size except in the 25 nodes dataset. The difference between the two models get smaller as the datasets get bigger.
- **Training time:** Regarding training time, models trained with circuit level data require less training time. This contrast accentuates in larger datasets, being noticeable in the 'full' dataset.

The following table presents some statistics of the performance metrics: mean, median and standard deviation.

Table 1: Model Performance Statistics		
Metric & Statistic	CIRCUIT MODEL	Q_VALUES MODEL
RMSE	Mean: 0.831556	Mean: 0.715895
	Median: 0.789147	Median: 0.674039
	Std: 0.141625	Std: 0.134631
MAPE	Mean: 0.351849	Mean: 0.347063
	Median: 0.315286	Median: 0.290289
	Std: 0.169690	Std: 0.169626

5.2 Multilayer Perceptron

Analogous to the XGBoost models, the multilayer perceptron models were evaluated in both the QUBO datasets and the circuit level data with the same metrics.

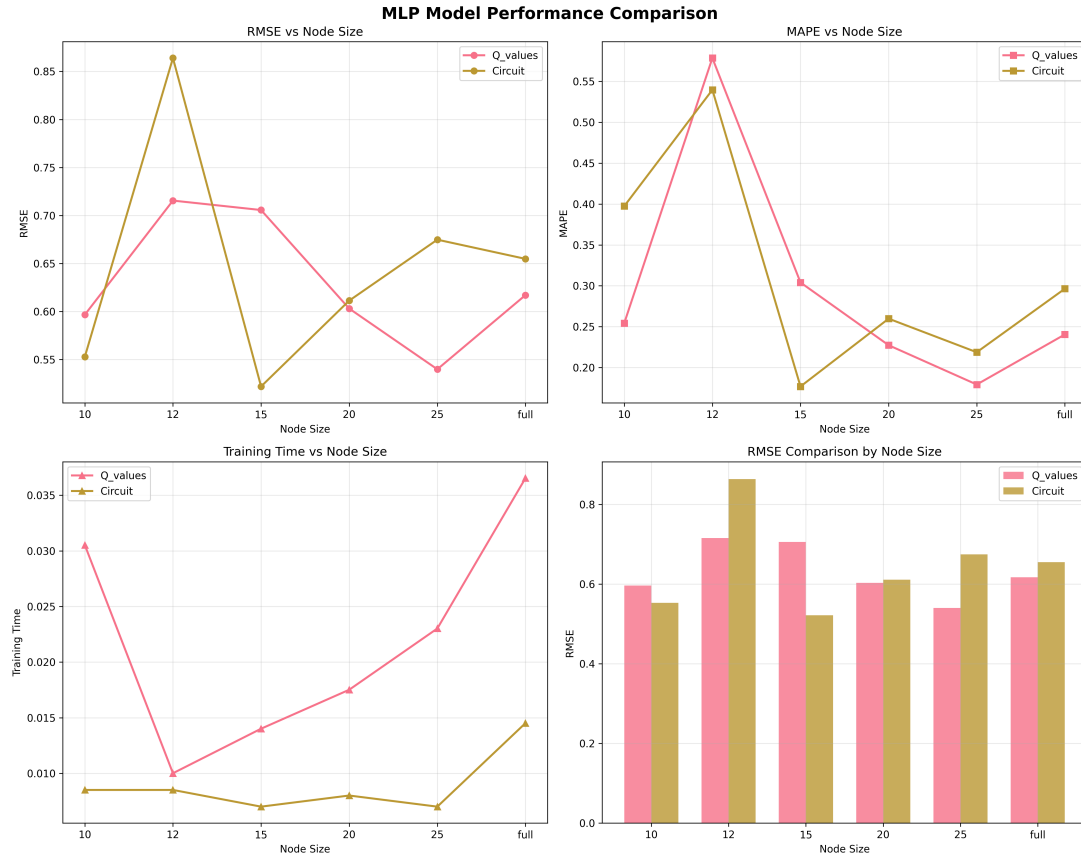


Figure 6: MLP Model Performance

The results of the evaluation are presented in Figure 6:

- **RMSE and MAPE:** Again while model performance generally improves with larger datasets, evidenced by decreasing error rates, the 12-node dataset represents a striking exception where accuracy significantly deteriorates, indicating unique modeling challenges. Furthermore, models using circuit-level data no longer consistently underperformed those using the QUBO matrix, out performing in both the 10 nodes and 15 nodes datasets. The performance gap between these two approaches still diminishes as dataset size increases.
- **Training time:** In terms of training time, models using circuit-level data are consistently faster to train. This efficiency gain grows more significant as dataset size increases, peaking with the 'full' dataset.

The following table presents some statistics of the performance metrics: mean, median and standard deviation.

Table 2: Model Performance Statistics		
Metric & Statistic	CIRCUIT MODEL	Q_VALUES MODEL
RMSE	Mean: 0.646668	Mean: 0.629696
	Median: 0.633137	Median: 0.610134
	Std: 0.121361	Std: 0.068056
MAPE	Mean: 0.314791	Mean: 0.297303
	Median: 0.278081	Median: 0.247378
	Std: 0.133421	Std: 0.143564

5.3 Graph Neural Networks

In this case, the same metrics were used. However, given the nature of the Graph Neural Networks, only the QUBO datasets were used.

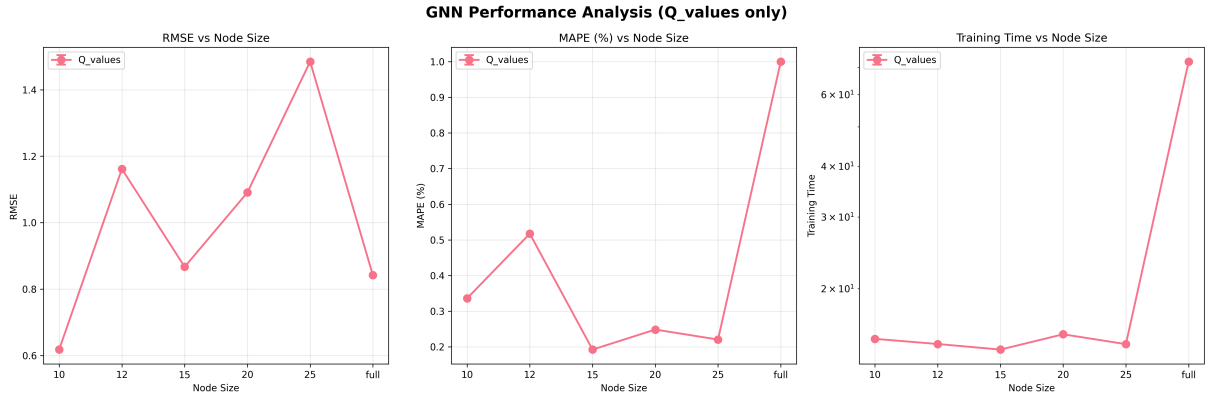


Figure 7: GNN Model Performance

As seen in Figure 7 Graph Neural Networks exhibit a consistent trend of decreasing error rates as node size increases, aligning with expected scaling behavior. However, when trained on the full dataset with padding applied, two critical issues emerge: First, training time becomes prohibitively long due to computational complexity. More significantly, the Mean Absolute Percentage Error (MAPE) spikes to absurdly high values (observed at approximately $1e+18$). This anomaly likely stems from division-by-zero or near-zero values in the MAPE calculation formula when actual values approach zero. Since such extreme values distort performance analysis and represent computational artifacts rather than meaningful error metrics, we substituted these outliers with a standardized value of 1 for realistic model comparison. This adjustment preserves data integrity while acknowledging the inherent limitations of MAPE when handling near-zero denominators.

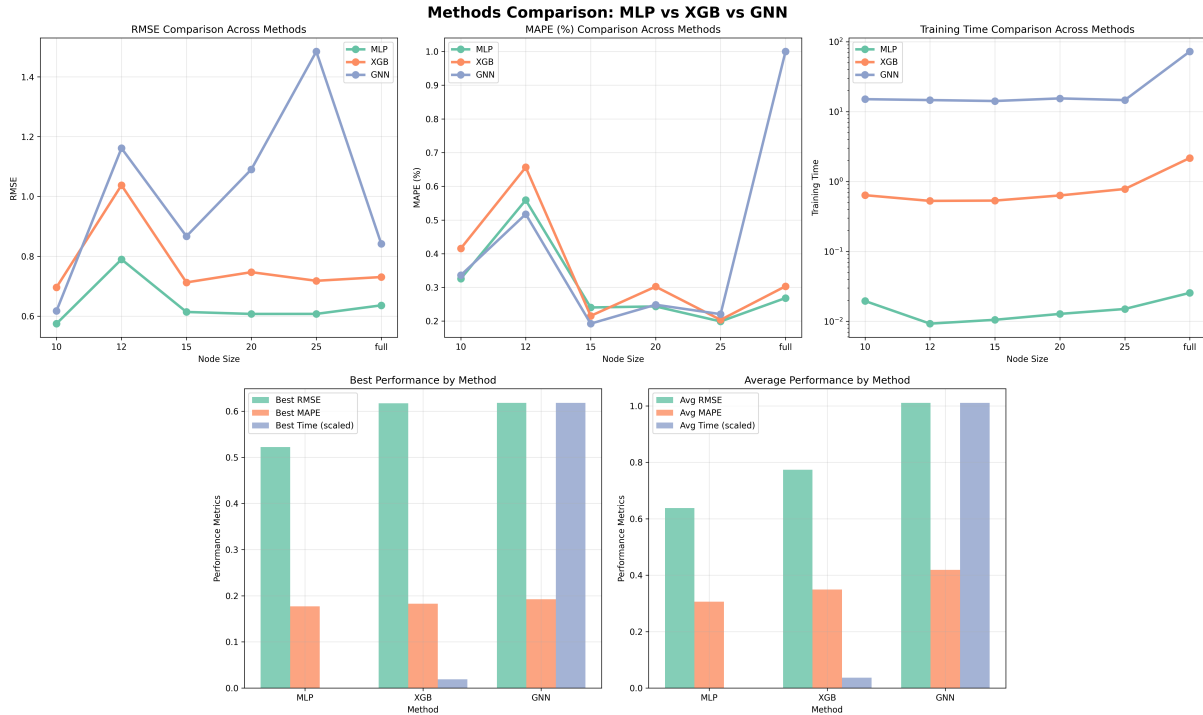
The following table presents some statistics of the performance metrics: mean, median and standard deviation.

Table 3: Model Performance Statistics

Metric & Statistic	Q_VALUES MODEL
RMSE	Mean: 0.872037
	Median: 0.749720
	Std: 0.309733
MAPE	Median: 0.207545

5.4 Model comparison

Finally, the performance metrics of the 3 types of models were also compared in the following graphs:



We can observe that Graph Neural Networks (GNNs) demonstrate the strongest raw predictive power in this analysis, consistently registering the lowest Mean Absolute Percentage Error (MAPE) across evaluations. However, this accuracy advantage comes with significant operational costs. GNNs exhibit a pronounced sensitivity to data padding, suffering a notable increase in error when this common preprocessing technique is applied. Furthermore, their training times are substantially longer compared to other models, creating a bottleneck for development and scaling.

In contrast, Multilayer Perceptrons (MLPs) present a highly efficient and robust alternative. They boast the absolute shortest training times, enabling faster experimentation and deployment. Crucially, while their peak MAPE might not always match the GNN's theoretical best, MLPs deliver accuracy that is remarkably close and highly competitive in the vast majority of cases. Given this combination of swift training and consistently strong, reliable performance, MLPs emerge as the most practical and desirable overall choice for balancing accuracy with computational efficiency.

5.5 Comparison with conventional QAOA approach

In this section, we examine how much time we can save by using the machine-learning model versus the traditional method for computing QAOA parameters.

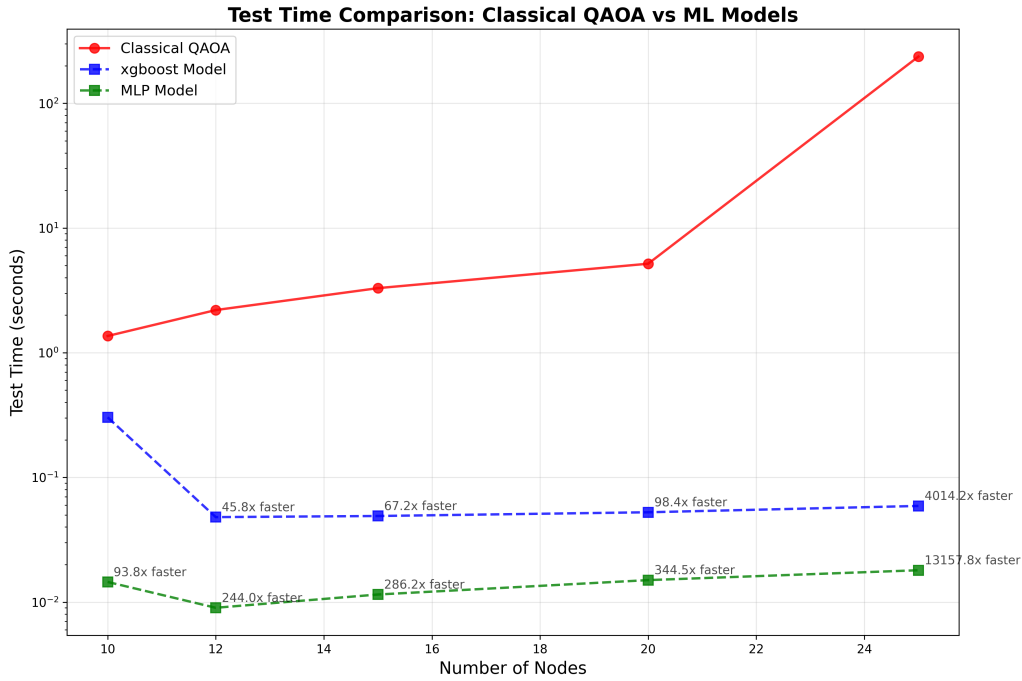


Figure 8: Time it takes each approach to solve a MaxCut instance

As seen in the Figure 8 the machine learning models (XGBoost and MLP) achieve dramatic speedups compared to classical QAOA across all problem sizes, as shown in the test time analysis. For smaller problem sizes (0.5% of the dataset), the ML models deliver results 45.8× to 4014.2× faster than classical QAOA. For larger problem sizes (9.3%), the speedup remains exceptionally high, ranging from 244.0× to 1315.8× faster.

These improvements reduce prediction times from hours/minutes (classical QAOA) to seconds or milliseconds (ML models), enabling real-time optimization. While model training demands initial computational effort, this one-time fixed cost is rapidly amortized in continuous deployment. Once operational, the persistent efficiency gains far outweigh the upfront training investment.

6

Conclusions and Futures Lines of Research

6.1 Conclusions

The methodology developed in this bachelor's thesis establishes an effective framework for reducing quantum resource consumption in optimization pipelines through supervised machine learning-parameterized QAOA. By leveraging classical machine learning models to predict optimal QAOA parameters, this approach achieves a significant reduction in quantum circuit evaluations—preserving valuable quantum computing resources while maintaining solution quality. This synergy between classical ML and quantum algorithms demonstrates that pre-optimization on classical systems should not be viewed as a computational overhead, but rather as an essential strategy for practical quantum advantage in the NISQ era.

Furthermore, regression techniques (including neural networks and ensemble methods) prove instrumental in capturing complex parameter landscapes across diverse problem instances. These models enable rapid, high-fidelity parameter initialization that would be infeasible through pure quantum optimization alone. The adaptability of this hybrid approach to varying problem sizes and its resilience to quantum noise positions ML-parametrized QAOA as a scalable pathway for real-world optimization, effectively bridging current computational limitations with emerging quantum capabilities.

6.2 Future lines of Research

Future developments of this work could explore several avenues to enhance the classical-quantum optimization pipeline. A primary direction involves extending the machine learning

parametrization framework to other quantum optimization algorithms beyond QAOA. Supervised learning models could be trained to predict optimal parameters for emerging quantum algorithms as they evolve, creating a scalable transfer-learning methodology for next-generation quantum processors.

Additionally, the integration of advanced neural architectures—such as graph neural networks tailored to quantum circuit topologies or meta-learners for rapid adaptation across problem classes—could dynamically refine parameter initialization. This would further reduce quantum resource consumption while improving solution accuracy. To ensure real-world applicability, future work should incorporate hardware-aware training that accounts for specific NISQ device noise profiles and qubit connectivity constraints. These advancements would accelerate the practical deployment of hybrid quantum-classical solvers across finance, logistics, and materials science, ultimately establishing ML-driven parametrization as a universal efficiency layer for quantum optimization.

References

- [1] Amazon Web Services. *¿En qué consiste la computación cuántica?* 2023. URL: <https://aws.amazon.com/es/what-is/quantum-computing/>.
- [2] Abhinav Anand et al. “A quantum computing view on unitary coupled cluster theory”. In: *Chemical Society Reviews* 51.5 (2022), pp. 1659–1684. ISSN: 0306-0012, 1460-4744. DOI: [10.1039/D1CS00932J](https://doi.org/10.1039/D1CS00932J).
- [3] Nancy Barraza et al. “Analog quantum approximate optimization algorithm”. In: *Quantum Science and Technology* 7.4 (Sept. 2022), p. 045035. ISSN: 2058-9565. DOI: [10.1088/2058-9565/ac91f0](https://doi.org/10.1088/2058-9565/ac91f0).
- [4] Kostas Blekos et al. “A review on Quantum Approximate Optimization Algorithm and its variants”. In: *Physics Reports* 1068 (June 2024), pp. 1–66. ISSN: 0370-1573. DOI: [10.1016/j.physrep.2024.03.002](https://doi.org/10.1016/j.physrep.2024.03.002). URL: <http://dx.doi.org/10.1016/j.physrep.2024.03.002>.
- [5] M. Cerezo et al. “Variational Quantum Algorithms”. In: *Nature Reviews Physics* 3.9 (Sept. 2021), pp. 625–644. ISSN: 2522-5820. DOI: [10.1038/s42254-021-00348-9](https://doi.org/10.1038/s42254-021-00348-9).
- [6] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 2016, pp. 785–794. ISBN: 9781450342322. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785). URL: <https://doi.org/10.1145/2939672.2939785>.
- [7] Francisco Chicano, Zakaria Abdelmoiz Dahi, and Gabriel Luque. “The Quantum Approximate Optimization Algorithm Can Require Exponential Time to Optimize Linear Functions”. In: *arXiv preprint arXiv:2505.06404* (2025). Accepted at the Quantum Optimization Workshop, Genetic and Evolutionary Computation Conference (GECCO 2025). DOI: [10.48550/arXiv.2505.06404](https://doi.org/10.48550/arXiv.2505.06404). URL: <https://arxiv.org/abs/2505.06404>.

- [8] Jaeho Choi and Joongheon Kim. “A Tutorial on Quantum Approximate Optimization Algorithm (QAOA): Fundamentals and Applications”. In: *2019 International Conference on Information and Communication Technology Convergence (ICTC)*. 2019, pp. 138–142. DOI: [10.1109/ICTC46691.2019.8939749](https://doi.org/10.1109/ICTC46691.2019.8939749).
- [9] Elizabeth Crosson et al. “Different strategies for optimization using the quantum adiabatic algorithm”. In: *arXiv preprint arXiv:1401.7320* (2014).
- [10] Vinicius Wilian D. Cruzeiro, Adrian Roitberg, and Nicolas C. Polfer. “Interactively Applying the Variational Method to the Dihydrogen Molecule: Exploring Bonding and Antibonding”. In: *Journal of Chemical Education* 93.9 (Sept. 2016), pp. 1578–1585. ISSN: 0021-9584, 1938-1328. DOI: [10.1021/acs.jchemed.6b00017](https://doi.org/10.1021/acs.jchemed.6b00017).
- [11] Zakaria Abdelmoiz Dahi et al. “Scalable Quantum Approximate Optimiser for Pseudo-Boolean Multi-objective Optimisation”. In: *Parallel Problem Solving from Nature – PPSN XVIII: 18th International Conference, PPSN 2024, Hagenberg, Austria, September 14–18, 2024, Proceedings, Part IV*. Hagenberg, Austria: Springer-Verlag, 2024, pp. 268–284. ISBN: 978-3-031-70084-2. DOI: [10.1007/978-3-031-70085-9_17](https://doi.org/10.1007/978-3-031-70085-9_17). URL: https://doi.org/10.1007/978-3-031-70085-9_17.
- [12] Prasanna Date et al. “Efficiently embedding QUBO problems on adiabatic quantum computers”. In: *Quantum Information Processing* 18.4 (Mar. 2019), p. 117. ISSN: 1573-1332. DOI: [10.1007/s11128-019-2236-3](https://doi.org/10.1007/s11128-019-2236-3).
- [13] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. “A quantum approximate optimization algorithm”. In: *arXiv preprint arXiv:1411.4028* (2014).
- [14] Dmitry A. Fedorov et al. “VQE method: A short survey and recent developments”. In: *Materials Theory* 6.1 (Jan. 2022), p. 2. ISSN: 2509-8012. DOI: [10.1186/s41313-021-00032-6](https://doi.org/10.1186/s41313-021-00032-6).
- [15] Matthias Fey and Jan E. Lenssen. “Fast Graph Representation Learning with PyTorch Geometric”. In: *arXiv preprint arXiv:1903.02428* (2019).
- [16] GeeksforGeeks. *Graph Neural Networks with PyTorch*. Accessed: August 15, 2023. 2023. URL: <https://www.geeksforgeeks.org/graph-neural-networks-with-pytorch/> (visited on 08/15/2023).

- [17] Fred Glover, Gary Kochenberger, and Yu Du. “Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models”. In: (), p. 46.
- [18] Michel X Goemans and David P Williamson. “Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming”. In: *Journal of the ACM (JACM)* 42.6 (1995), pp. 1115–1145.
- [19] Gian Giacomo Guerreschi and Akira Matsuura. “QAOA for max-cut requires hundreds of qubits for quantum speed-up”. In: *Scientific reports* 9.1 (2019), p. 6903.
- [20] Stas Hadfield. “Quantum algorithms for scientific computing and approximate optimization”. In: *arXiv preprint arXiv:1805.03265* (2018).
- [21] Stas Hadfield et al. “From the quantum approximate optimization algorithm to a quantum alternating operator ansatz”. In: *Algorithms* 12.2 (2019), p. 34.
- [22] IBM Quantum. *Solve utility-scale quantum optimization problems*. Accessed: 2023-11-01. IBM. 2023. URL: <https://learning.quantum.ibm.com/tutorial/quantum-approximate-optimization-algorithm> (visited on 11/01/2023).
- [23] Abhinav Kandala et al. “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets”. In: *Nature* 549.7671 (Sept. 2017), pp. 242–246. ISSN: 1476-4687. DOI: [10.1038/nature23879](https://doi.org/10.1038/nature23879).
- [24] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *arXiv preprint arXiv:1609.02907* (2016).
- [25] Wolfgang Lechner. “Quantum Approximate Optimization With Parallelizable Gates”. In: *IEEE Transactions on Quantum Engineering* 1 (2020), pp. 1–6. ISSN: 2689-1808. DOI: [10.1109/TQE.2020.3034798](https://doi.org/10.1109/TQE.2020.3034798).
- [26] Wolfgang Lechner, Philipp Hauke, and Peter Zoller. “A quantum annealing architecture with all-to-all connectivity from local interactions”. In: *Science Advances* 1.9 (Oct. 2015), e1500838. DOI: [10.1126/sciadv.1500838](https://doi.org/10.1126/sciadv.1500838).
- [27] Martin Leib, Peter Zoller, and Wolfgang Lechner. “A transmon quantum annealer: Decomposing many-body Ising constraints into pair interactions”. In: *Quantum Science and Technology* 1.1 (Dec. 2016), p. 015008. ISSN: 2058-9565. DOI: [10.1088/2058-9565/1/1/015008](https://doi.org/10.1088/2058-9565/1/1/015008).

- [28] H. Lizano. *Computación Cuántica*. Universidad de Costa Rica. 2023. URL: <https://acaducr.ucr.ac.cr/computacion-cuantica/>.
- [29] Bas Lodewijks. “Mapping NP-hard and NP-complete optimisation problems to Quadratic Unconstrained Binary Optimisation problems”. In: (Aug. 2020).
- [30] Andrew Lucas. “Ising formulations of many NP problems”. In: *Frontiers in Physics* 2 (2014). ISSN: 2296-424X.
- [31] Catherine C McGeoch. “Adiabatic quantum computation and quantum annealing: Theory and practice”. In: *Synthesis Lectures on Quantum Computing* 5.2 (2014), pp. 1–93.
- [32] C. Moussa, H. Wang, T. Back, et al. “Unsupervised strategies for identifying optimal parameters in Quantum Optimization Algorithm”. In: *EPJ Quantum Technology* 9.11 (2022), p. 11. DOI: [10.1140/epjqt/s40507-022-00131-4](https://doi.org/10.1140/epjqt/s40507-022-00131-4). URL: <https://doi.org/10.1140/epjqt/s40507-022-00131-4>.
- [33] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [34] John Preskill. “Quantum computing in the nisc era and beyond”. In: *Quantum* 2 (2018), p. 79.
- [35] Eleanor G Rieffel et al. “From ansätze to z-gates: a NASA view of quantum computing”. In: *arXiv preprint arXiv:1905.02860* (2019).
- [36] Ruslan Shaydulin et al. “A hybrid approach for solving optimization problems on small quantum computers”. In: *Computer* 52.6 (2019), pp. 18–26.
- [37] Javier Mas Sole. *Curso básico de computación cuántica*. <https://talentq-es.github.io/TalentQ-CICC/docs/index.html>. TalentQ (<https://www.talentq.es/es-es/>). 2024.
- [38] Zsolt Tabi et al. “Quantum Optimization for the Graph Coloring Problem with Space-Efficient Embedding”. In: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Oct. 2020, pp. 56–62. DOI: [10.1109/QCE49297.2020.00018](https://doi.org/10.1109/QCE49297.2020.00018).
- [39] Jules Tilly et al. “The Variational Quantum Eigensolver: A review of methods and best practices”. In: *Physics Reports* 986 (Nov. 2022), pp. 1–128. ISSN: 0370-1573. DOI: [10.1016/j.physrep.2022.08.003](https://doi.org/10.1016/j.physrep.2022.08.003).

- [40] Universidad de Empresa. *Explorando la Computación Cuántica*. 2023. URL: <https://ude.edu.uy/que-es-la-computacion-cuantica/>.
- [41] Manuela Weigold et al. “Patterns for Hybrid Quantum Algorithms”. In: *Service-Oriented Computing*. Ed. by Johanna Barzen. Vol. 1429. Cham: Springer International Publishing, 2021, pp. 34–51. ISBN: 978-3-030-87567-1, 978-3-030-87568-8. DOI: [10.1007/978-3-030-87568-8_2](https://doi.org/10.1007/978-3-030-87568-8_2).
- [42] Wikipedia Contributors. *Computación cuántica*. Wikipedia. 2023. URL: https://es.wikipedia.org/wiki/Computaci%C3%B3n_cu%C3%A1ntica.
- [43] Leo Zhou et al. “Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices”. In: *arXiv preprint arXiv:1812.01041* (2018).

Appendix A

Installation

Manual

To perform the installation, Python 3.11.7 or higher must be installed, along with the following packages:

- numpy
- Qiskit
- Pandas
- Scikit-learn
- Pytorch

It is recommended to install within a virtual environment and execute the following command:
`pip install -r requirements.txt`

This will automatically install all the packages mentioned above.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga